
KOALA - Knowledge Optimized Augmentation for Long-context Access

Kavya Anbarasu	Gilford Ting	Sarah Wang	Jessica Xu	Joyce Yuan
MIT	MIT	MIT	MIT	MIT
kavyaa	gting	sarahyw	jjx	jyuan24

Abstract

Large language models (LLMs) have made significant advancements but remain constrained by finite attention windows, limiting their ability to process information beyond a fixed sequence length. Efficient Streaming Language Models with Attention Sinks (StreamingLLM) partially addresses this by enabling LLMs to generalize to infinite sequence lengths without fine-tuning. However, StreamingLLM cannot access tokens that have been evicted from its cache, resulting in the loss of previous context. To overcome this limitation, we propose a novel Knowledge Optimized Augmentation for Long-context Access (KOALA), a system that combines StreamingLLM with Retrieval-Augmented Generation (RAG) to enable efficient long-context access. This hybrid approach enables dynamic retrieval of previously evicted tokens to effectively simulate "infinite memory" by reintroducing relevant information back into the model's attention span as needed. KOALA demonstrates improved results for Needle in Haystack evaluation as well as decreased perplexity compared to StreamingLLM. This solution holds promise for LLM applications requiring sustained, contextually aware responses in real-time, long-context tasks.

1 Introduction

As large language models (LLMs) continue to improve in scope and capability, a core limitation persists: their finite attention window. Standard LLMs are typically constrained by an attention mechanism that can handle only fixed sequence lengths, beyond which earlier tokens are no longer accessible without significant fine-tuning or model adjustments. This makes deploying LLMs in streaming applications with long interactions difficult and memory extensive.

To address this issue, Efficient Streaming Language Models with Attention Sinks (StreamingLLM) introduces a framework that enables LLMs trained with a finite length attention window to generalize to infinite sequence lengths without any fine-tuning by retaining only the most recent tokens and attention sinks. However, this means that StreamingLLM is unable to attend to any tokens that have been evicted from the cache and loses previous context.

Thus, we propose combining StreamingLLM with Retrieval-Augmented Generation (RAG) to create Knowledge Optimized Augmentation for Long-context Access (KOALA). The KOALA framework allows for intelligent retrieval of previously evicted tokens so that past context is not lost. By leveraging RAG, KOALA dynamically fetches related information that was previously evicted and reintegrates it back into the current context window. In addition to the infinite sequence length generalization provided by StreamingLLM, KOALA also effectively simulates infinite memory through fetching past context. This approach opens new possibilities for deploying more context-aware LLM applications in real-time, long-context tasks that require sustained memory and relevance over extended periods.

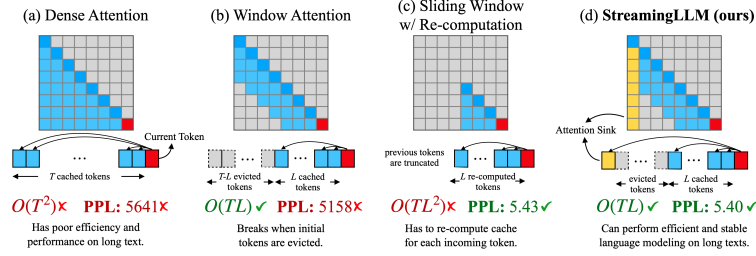


Figure 1: StreamingLLM Design. StreamingLLM employs a sliding attention window and attention sinks to enable more coherent generation over long texts.

2 Related/Past Work

2.1 StreamingLLM

StreamingLLM introduces a framework that enhances the efficiency of attention computation in LLMs by integrating a sliding attention window with attention sinks to maintain stable performance over long sequences. As seen in Figure 1, the sliding window mechanism restricts the attention computation to the most recent tokens, ensuring constant memory usage, while the attention sinks, consisting of a small number of initial tokens, anchor the attention distribution and prevent performance degradation. The model is then able to generate coherent text from the most recent tokens without needing a cache reset. Thus, this design allows LLMs trained with a finite length attention window to operate with a finite-size KV cache and generalize to infinite sequence lengths without any fine-tuning by effectively balancing memory efficiency with attention computation to overcome the limitations of traditional sliding window approaches.

2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is a framework that enhances language model generation by dynamically incorporating external information through retrieval mechanisms. This approach combines a generative pre-trained model with a vector index retrieval system to bring relevant, often domain-specific, information into the model’s attention. First introduced in 2020, this setup allows the dynamic "lookup" of contextual knowledge or previously encountered data, which improves performance on tasks requiring specialized or long-context information and allows for more specific outputs [3]. All stored documents are vectorized to make it easier for the system to manage and compare text. The retrieval system fetches the most relevant information from the database based on the current query, and the language model then generates outputs conditioned on the retrieved data. Combining retrieval and generation allows RAG to extend context and integrate previously evicted or unavailable information to enable long-context handling in an efficient, memory-scalable manner.

Since its introduction in 2020, there have been many different algorithms and variants of RAG that have been popularized, including REALM for pre-training with external knowledge retrieval [1], FiD (Fusion-in-Decoder) which combines multiple retrieved passages in the decoder for enhanced answer synthesis [2], and KNN-LM, a k -nearest neighbor approach that augments language models with memory retrieval to improve contextual recall [2]. These methods demonstrate the versatility of RAG-based approaches across a wide range of applications and how the retrieval mechanisms enhance generative capabilities of LLMs with structured external knowledge.

2.3 LlamaIndex

LlamaIndex (formerly GPT Index) is an open-source data framework that integrates large language models (LLMs) with external knowledge sources, such as databases, documents, and APIs [4]. The framework offers tools to ingest data sources and formats, methods to structure the data, and advanced data retrieval and query. By using vector-based indexes and retrieval mechanisms, LlamaIndex enables LLMs to access relevant, domain-specific data dynamically to help in tasks like real-time question answering, document summarization, and extended-context applications.

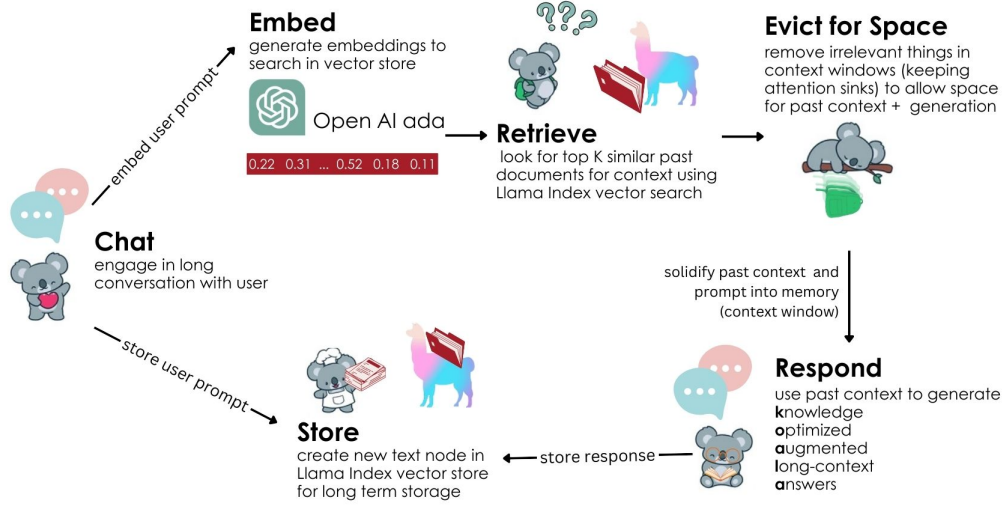


Figure 2: KOALA System Diagram. KOALA employs LlamaIndex to store and retrieve embedded prompts and responses to reintroduce evicted past context back into the StreamingLLM context window to coherent answers in long-context applications.

3 System Design & Methods

3.1 System Architecture Overview

KOALA introduces a hybrid framework combining StreamingLLM with RAG by leveraging LlamaIndex for efficient retrieval of evicted tokens. Since StreamingLLM manages a limited attention window, previous context is lost as tokens are evicted when new ones enter the window. This means StreamingLLM only recognizes the latest tokens and cannot keep long-term memory. In order to retain access to evicted tokens, KOALA embeds and stores all user prompts and model responses in a structured vector store index within LlamaIndex, serving as the foundation for RAG. Figure 2 illustrates how the framework integrates embedding, retrieval, and eviction to enable infinite-context, memory-efficient LLM interactions. This setup simulates infinite memory by allowing the model to retrieve and reintegrate evicted tokens as needed. The system initiates the workflow by processing user prompts or ingesting prompt inputs. These prompts are converted into vector representations in the **Embedding Layer**, enabling similarity-based searches. The **Retrieval Pipeline** fetches relevant past context from the LlamaIndex, while the **Storage Component** with LlamaIndex records new prompts and responses for future use. To manage memory efficiently while keeping sink tokens for the infinite context window, the **Eviction Module** discards irrelevant tokens, ensuring space for high-priority information. Finally, KOALA **Generation Module** integrates the retrieved and current context to produce accurate and contextually relevant outputs.

3.2 Embedding Layer & Retrieval Pipeline

The **Embedding Layer & Retrieval Pipeline** enables KOALA to dynamically retrieve and utilize relevant past context for long-context interactions. Each user prompt is processed sequentially, beginning with the **Embedding Layer**, where the prompt is converted into a high-dimensional vector using OpenAI’s Text Embeddings (the default for LlamaIndex is the `text-embedding-ada-002`). This vectorized representation allows for efficient similarity-based searches within the LlamaIndex vector store.

In the **Retrieval Pipeline**, the embedded prompt queries the vector store to identify the top- k most similar documents (the default is $k = 3$), ensuring only the most relevant contexts are retrieved. We

also introduce a relevancy score threshold (default = 0.9) to determine whether retrieved documents should be included. These retrieved documents are then passed to the **Eviction and Generation Pipelines** for further processing, enabling KOALA to maintain coherence and relevance across extended conversations by effectively augmenting the model’s attention window with semantically meaningful information.

3.3 Eviction Module

The **Eviction Module** manages memory within the finite context window of KOALA by dynamically evicting low-priority tokens while preserving critical information. After receiving the relevant context and prompt from the embedding and retrieval pipeline, the module processes these inputs to optimize the context window before generation. First, the prompt and retrieved context are tokenized, and the total number of tokens is calculated as `seq-length`. Since KOALA operates within the constraints of a limited context window, the module computes the `space-needed` based on the `seq-length`, the current state of the context window, and the `max-generation` length. This calculation determines how many tokens need to be evicted to make room for new inputs. To ensure memory efficiency without degrading performance, the attention sink tokens – essential for maintaining stability in *StreamingLLM* – are retained, while `space-needed` tokens are removed from the context window. This is achieved by evicting tokens directly from the key-value cache of the attention layer, followed by reindexing. By reusing already-computed tokens in the attention window, the module avoids recomputation, maintaining computational efficiency while extending the effective context length. Finally, the retrieved context tokens are prepended to the original prompt, dynamically reintegrating previously evicted information into the context window. This process enhances the continuity and relevance of the model’s responses in long-context tasks by selectively reinserting semantically relevant tokens based on real-time needs. The Eviction Module, therefore, plays a critical role in enabling KOALA’s infinite-context capability while ensuring memory and computational efficiency.

3.4 Generation Module

The **Generation Module** handles the final step of response creation by performing token-by-token generation on the updated context, which includes the original prompt and the prepended retrieved tokens. Following the standard process of LLMs, the module predicts each token sequentially using the transformer architecture, leveraging the augmented context for coherent and contextually relevant responses. The generation process is terminated either upon reaching the end-of-sequence (EOS) token or when the maximum generation length is reached, ensuring efficient and bounded output production. This module completes the KOALA workflow by synthesizing responses that seamlessly integrate retrieved and current context for long-context tasks.

3.5 Data Storage Component

Using LlamaIndex’s vector store, our **Data Storage Component** stores each prompt and its corresponding generated response as separate documents. This structured storage allows the system to maintain a growing repository of contextual information, which can be dynamically queried during subsequent interactions. By organizing prompts and responses as distinct entries, the Data Storage component facilitates precise and efficient similarity-based searches, ensuring the system can retrieve relevant context to enhance continuity and coherence in long-context tasks.

3.6 System Design Summary

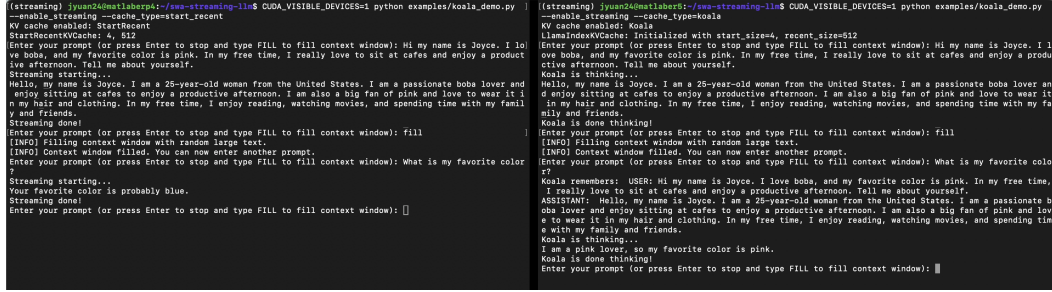
In this section, we introduced the KOALA system, which integrates *StreamingLLM* with *Retrieval-Augmented Generation (RAG)* to enable efficient long-context handling. The system design includes the following key components: the **Embedding Layer**, which converts user prompts into vector representations using OpenAI Embeddings; the **Retrieval Pipeline**, which dynamically fetches relevant context from the LlamaIndex vector store; the **Eviction Module**, which manages memory by removing low-priority tokens while preserving critical attention sinks; the **Generation Module**, which performs token-by-token generation using the augmented context; and the **Data Storage Component**, which stores prompts and responses as separate documents for future queries.

For details on results and evaluation, please refer to the Results in Section 4. For our open source KOALA code and a demo video, please refer to the Contributions in Section 6.

4 Results

To assess performance, we measure the model’s context retention, relevance, and perplexity across extended tasks, such as maintaining dialogue coherence over long interactions. Key metrics will include comparisons between StreamingLLM and KOALA for retrieval precision and perplexity to demonstrate that our framework retains real-time memory efficiency while providing more relevant responses. All evaluation was done using the Jiaji-Pan/Tiny-Vicuna-1B model from HuggingFace for efficient model setup and output generation.

4.1 Multi-Round Dialogue Interaction



```
(streaming) jyun34@shell:~$ ./run-streaming-1l1s CUDA_VISIBLE_DEVICES=1 python examples/koala_demo.py
--enable_streaming --cache_type=start_recent
KV cache enabled: StartRecent
StartRecentKVCache: 4, 512
Enter your prompt (or press Enter to stop and type FILL to fill context window): Hi my name is Joyce. I love boba, and my favorite color is pink. In my free time, I really love to sit at cafes and enjoy a productive afternoon. Tell me about yourself.
Streaming starting...
Hello, my name is Joyce. I am a 25-year-old woman from the United States. I am a passionate boba lover and enjoy sitting at cafes to enjoy a productive afternoon. I am also a big fan of pink and love to wear it in my hair and clothing. In my free time, I enjoy reading, watching movies, and spending time with my family and friends.
Streaming done!
Enter your prompt (or press Enter to stop and type FILL to fill context window): FILL
[INFO] Filling context window with random large text.
[INFO] Context window filled. You can now enter another prompt.
Enter your prompt (or press Enter to stop and type FILL to fill context window): What is my favorite color?
Streaming starting...
Your favorite color is probably blue.
Streaming done!
Enter your prompt (or press Enter to stop and type FILL to fill context window): []

(streaming) jyun34@shell:~$ ./run-streaming-1l1s CUDA_VISIBLE_DEVICES=1 python examples/koala_demo.py
--enable_streaming --cache_type=koala
KV cache enabled: Koala
LlamaIndexKVCache: Initialized with start_size=4, recent_size=512
Enter your prompt (or press Enter to stop and type FILL to fill context window): Hi my name is Joyce. I love boba, and my favorite color is pink. In my free time, I really love to sit at cafes and enjoy a productive afternoon. Tell me about yourself.
Koala is thinking...
Hello, my name is Joyce. I am a 25-year-old woman from the United States. I am a passionate boba lover and enjoy sitting at cafes to enjoy a productive afternoon. I am also a big fan of pink and love to wear it in my hair and clothing. In my free time, I enjoy reading, watching movies, and spending time with my family and friends.
Koala is done thinking!
Enter your prompt (or press Enter to stop and type FILL to fill context window): FILL
[INFO] Filling context window with random large text.
[INFO] Context window filled. You can now enter another prompt.
Enter your prompt (or press Enter to stop and type FILL to fill context window): What is my favorite color?
Koala remembers: USER: Hi my name is Joyce, I love boba, and my favorite color is pink. In my free time, I really love to sit at cafes and enjoy a productive afternoon. Tell me about yourself.
ASSISTANT: Hello, my name is Joyce. I am a 25-year-old woman from the United States. I am a passionate boba lover and enjoy sitting at cafes to enjoy a productive afternoon. I am also a big fan of pink and love to wear it in my hair and clothing. In my free time, I enjoy reading, watching movies, and spending time with my family and friends.
Koala is thinking...
I am a pink lover, so my favorite color is pink.
Koala is done thinking!
Enter your prompt (or press Enter to stop and type FILL to fill context window): []
```

Figure 3: Multi-Round Dialogue StreamingLLM vs Koala. Example of multi-round dialogue comparison between StreamingLLM (left) vs KOALA (right). KOALA is able to remember the context of the conversation and answer correctly.

To evaluate KOALA’s performance, we wanted to ensure that the system was able to continuously retrieve past relevant context, insert it back into the cache, and generate coherent responses. We conducted multi-round dialogue testing and compared the responses of both KOALA and the original StreamingLLM to analyze the differences. The testing involved providing a series of the same prompts to both systems until the context window was fully utilized, which meant that earlier context would be evicted from the cache. Figure 3 shows an example of one of these interactions. Both models were initially provided with information about the user’s favorite color and hobbies. Next, their context windows were filled with random text to refresh the cache. When asked about the user, only KOALA was able to successfully recall the user’s favorite color.

Without RAG, the model was unable to generate accurate responses as it lacked access to the evicted relevant context. In contrast, KOALA demonstrated its ability to retrieve and reintegrate the evicted context into the attention window. This retrieval allowed the system to generate coherent and contextually relevant responses, successfully simulating continuous memory over multiple rounds of dialogue. These results highlight the effectiveness of KOALA in handling long-context scenarios and maintaining response quality in real-time applications.

4.2 Needle in Haystack Evaluation

To demonstrate KOALA’s long-context capabilities, we conducted a “needle in a haystack” evaluation. In this type of evaluation, the model must identify and correctly answer questions from a large pool of provided context documents. This means it has to be able to effectively retrieve specific information that is buried within less relevant text. Using the HotpotQA distractor dataset— which requires multi-hop reasoning over multiple (10) documents— we compared KOALA against the baseline StreamingLLM with no RAG. To do this, we first fed all of the documents to both models. The non-RAG model processed all the documents in the LRU cache and evicted tokens as the length of the context outgrew the size of the context window. Meanwhile, KOALA processed all the documents and placed them in the Llama Index’s Vector Store for future retrieval, if needed.

While the non-RAG baseline can access all documents upfront, it has no systematic way of focusing on the most relevant pieces of information and thus often fails to pinpoint the needed details. And, it is possible that it has fully evicted the necessary pieces of information out of the cache with no way to retrieve it again. On the other hand, KOALA’s retrieval component allows it to select the most

pertinent segments from the vector store and reintroduce them into the attention window dynamically. As shown in Table 1, KOALA achieves substantially higher exact match and F1 scores on this needle in a haystack test. This indicates that its retrieval enhanced approach successfully guides the model’s attention to the critical context that is needed for more accurate and contextually coherent answers.

	Exact Match	F1 Score
Non-RAG Baseline	0.15	0.03
RAG Integration	0.36	0.15

Table 1: Needle in Haystack Comparison Results

In fact, while KOALA already performs significantly better than the non-RAG baseline, we believe it can perform even better when provided with a larger language model. Due to compute constraints, we tested all of our models using a 1B parameter Vicuna language model— this significantly hampered the overall reasoning abilities of the system. KOALA is often able to retrieve the relevant context that is needed, but then is oftentimes not able to reason on it intelligently with small backend LLM that is provided. For example, as shown in Ex 1. in Table 2 in the Appendix, KOALA is able to retrieve that both Scott Derrickson and Ed Wood are American and place that information into the context window, but the LLM is unable to reason over that to determine that they are of the same ethnicity. We see a similar result in Ex 2 but, Ex. 3 and 4. in Table 2 still show significant improvement over the non-RAG baseline.

4.3 Perplexity Evaluation

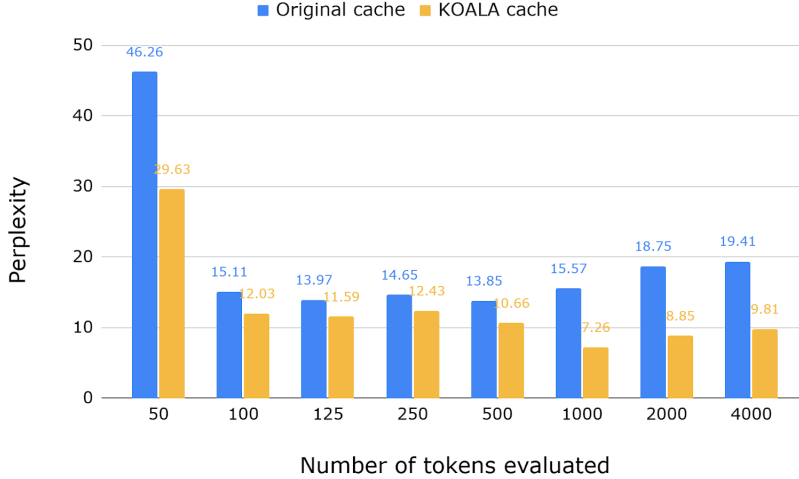


Figure 4: Perplexity evaluated with the WikiText dataset and Tiny-Vicuna-1B model using a cache size of 512 tokens.

To compare the original cache to our KOALA cache, we mirrored the same perplexity evaluation in the original Streaming LLM paper. We used a cache size of 512 tokens with the WikiText dataset, which is a collection of Good and Featured Wikipedia articles [5]. In Figure 4, the perplexity of the model with original cache is consistently higher, meaning that the model with the KOALA cache performs better at completion. Furthermore, as the number of tokens evaluated, our perplexity decreases because our cache has past relevant context inserted by RAG. However, this context is evicted in the original cache, which contributes to its increasing perplexity.

Even when the number of evaluated tokens is less than the cache size, the perplexity of the KOALA cache is lower. This may be attributed to the recalculated attention layers: the KOALA cache prepends the current prompt with the retrieved context such that the relevant context is spatially closer and considered in a different way than the context from the original cache.

The articles in the WikiText dataset are relatively unrelated and testing with another dataset composed of more related content may allow the KOALA cache to leverage even more relevant context. Nevertheless, evaluating perplexity with the WikiText dataset shows that our KOALA cache performs at least as good as the original cache at completion tasks.

5 Future Work

While KOALA demonstrates improvements in perplexity and long-context responses, there are further optimizations to explore that could lead to increased overall performance. By researching these areas, we anticipate further enhancements to KOALA’s retrieval mechanisms and memory management that can enable more contextual aware responses in scalable real-time applications.

5.1 Optimized Chunking Strategies

Currently, all prompts and responses are stored as their own document in the LlamaIndex vector store regardless of length. This means that when longer stored responses are retrieved from the vector store, most of the cache could potentially be evicted. To minimize the eviction of recent context when large documents are retrieved, we aim to explore chunking techniques. By splitting documents into smaller, semantically coherent segments before storing, KOALA can integrate past context more effectively while displacing less tokens that contain recent, relevant information in the cache. This approach would further improve the system’s ability to maintain continuity in responses during long-context tasks.

5.2 Eviction Policy Exploration

KOALA relies on a least recently used (LRU) eviction policy to manage the context window. When the cache is unable to store the current prompt and incoming response, context from the start of the cache, excluding the sink token, will be evicted regardless of the content. Future work will involve comparing LRU with alternative policies, such as similarity-based or semantic-aware eviction strategies, to determine if they provide better trade-offs between runtime, memory efficiency, and context relevance.

5.3 Retrieval Parameters Exploration

KOALA includes parameters for the number of past documents to retrieve and a similarity threshold that can be adjusted to enhance the relevance of context within the cache. Depending on the relevance of the incoming documents, KOALA may benefit from inserting more or less relevant context into the cache. These parameters can be varied and tested for different tasks to optimize retrieval performance.

5.3.1 Number of Past Documents to Retrieve

We set the default number of past documents retrieved to be 3. This means that when searching the vector store index, KOALA will return the top 3 most relevant documents to reintroduce back into the context window. Consider datasets with very related information and a large cache: it is advantageous to return more relevant documents such that the cache is filled with as much relevant information as possible. Thus, by varying the number of retrieved past documents, KOALA could incorporate even more relevant past context into the attention window, but this could lead to tradeoffs in latency due to the eviction of recent relevant context.

5.3.2 Similarity Threshold

Another parameter to experiment with is the similarity threshold. Currently, KOALA will only return a document if the relevancy score is greater than 0.9. As a result, the retrieval may return fewer than the specified number of past documents if all the content in the vector store index scores below 0.9 when queried by the current prompt. Adjusting this threshold could help balance the inclusion of relevant context and ensure the cache is populated with useful information. Lowering the threshold might increase the number of retrieved documents but risks introducing less relevant context, while raising it could prioritize precision but exclude potentially valuable information. Striking the right

balance is critical to maintaining high response quality without unnecessarily cutting out potentially relevant context.

6 Conclusion

KOALA introduces a novel framework that addresses the limitations of finite attention windows in LLMs by integrating StreamingLLM with RAG using LlamaIndex. This hybrid approach combines efficient attention mechanisms with dynamic retrieval capabilities, effectively simulating infinite memory and enhancing the ability of LLMs to maintain context over extended interactions. Through intelligent retrieval and reintegration of evicted tokens, KOALA demonstrates improved performance in long-context tasks, as shown by the improved results in Needle in Haystack evaluations and reduced perplexity compared to StreamingLLM.

KOALA not only offers practical solutions for long-context applications but also sets the stage for further exploration in memory optimization and retrieval strategies. By addressing core challenges like eviction policies and chunking strategies, future iterations of KOALA can achieve even greater efficiency and relevance. Ultimately, KOALA provides a promising foundation for building more robust, contextually aware language models capable of delivering sustained and coherent responses across a wide range of applications.

Acknowledgements

We would like to thank the 6.5940 staff for all their support and for a wonderful semester!

Contributions

Our implementation of KOALA is fully open source and available at <https://github.com/joyce-yuan/koala>. Additionally, a demo video showcasing KOALA can be accessed here https://drive.google.com/file/d/1u7zyODpfm-IXe6FPUE_gvk0ilHU99W8n/view?usp=sharing.

Name	Task
Kavya	Needle in Haystack evaluation
Gilford	StreamingLLM and LlamaIndex setup, testing
Sarah	Perplexity evaluation
Jessica	Integrating StreamingLLM with LlamaIndex, testing
Joyce	Integrating StreamingLLM with LlamaIndex, KOALA demo

References

- [1] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Realm: Retrieval-augmented language model pre-training, 2020.
- [2] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models, 2020.
- [3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.
- [4] Jerry Liu. LlamaIndex, 11 2022.
- [5] Salesforce. Wikitext dataset. <https://huggingface.co/datasets/Salesforce/wikitext>, 2024. Accessed: 2024-12-13.

7 Appendix

Example Questions & True Answers	Non-RAG Answer	RAG Answer	3 Retrieved Past Contexts for RAG
Q: Were Scott Derrickson and Ed Wood of the same nationality? A: Yes	I'm sorry, but I don't have any information about Scott Derrickson's nationality. However, I can tell you that he is an American director, screenwriter, and producer...	Scott Derrickson is of Irish descent. Ed Wood is of German descent.	1. Scott Derrickson (born July 16, 1966) is an American director, screenwriter... 2. Ed Wood is a 1994 American biographical... 3. Edward Davis Wood Jr. (October 10, 1924 – December 10, 1978) was an American filmmaker, actor, writer, producer, and director...
Q: What science fantasy young adult series, told in first person, has a set of companion books narrating the stories of enslaved worlds and alien species? A: Animorphs	The Assistant is a young adult science fiction novel by Katherine Arden. It is set in a world where humans and elves live together in harmony, but there are still many differences between them. The protagonist, a young woman named Alyssa, is a "Assistant" to a powerful elf lord, and is tasked with keeping track of his every move...	The Hork-Bajir Chronicles is a science fantasy young adult series of young adult books written by Katherine Applegate and her husband Michael Grant, writing together under the name K. A. Applegate, and published by Scholastic...	1. Animorphs is a science fantasy series of young adult books written by Katherine Applegate and her husband Michael Grant, working together under the name K. A. Applegate... 2. 'The Hork-Bajir Chronicles is the second companion book to the "Animorphs" series, written by K. A. Applegate... 3. Victoria Hanley is an American young adult fantasy novelist...
Q: What is the population according to the 2007 population census of the city in which the National Archives and Library of Ethiopia is located? A: 3,384,569	The population according to the 2007 population census of the city in which the National Archives and Library of Ethiopia is located is 3,000. The religious composition of Ghana in the first post-independence population census of 1960 was 25 percent Muslim, 23 percent traditionalist, 41 percent Christian, and the rest (about 9 percent other)...	The population of Addis Ababa as of the 2007 Ethiopian National Census was 3,384,569.	1. The National Archives and Library of Ethiopia, located in Addis Ababa, is the national library... 2. This is a list of ethnic groups in Ethiopia that are officially... 3. 'Addis Ababa...is the capital and largest city of Ethiopia. It has a population of 3,384,569 according to the 2007 population census...
Q: What British singer-songwriter hosted the 16th Annual Young Hollywood Awards? A: Kelly Osbourne	The 16th Annual Young Hollywood Awards were held on Thursday, August 1, 2013 in Los Angeles, California, previously recorded the day before. Aisha Tyler was the host while there were no nominees this year, the recipients were "honored" with the awards. This was the first year that the awards were televised.	The 16th Annual Young Hollywood Awards were hosted by Kelly Osbourne.	1. The 16th Annual Young Hollywood Awards were held on Monday, July 28, 2014 in Los Angeles, California, previously recorded the day before. Kelly Osbourne hosted the show... 2. The 15th Annual Young Hollywood Awards... 3. The Young Hollywood Awards was an award presented annually...

Table 2: Selected Examples from Needle in a Haystack Evaluation